main.py

```python
INF = 999999

def shortest_distance(graph, u, v):
    n = len(graph)

    # Floyd-Warshall
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if graph[i][k] + graph[k][j] < graph[i][j]:
                    graph[i][j] = graph[i][k] + graph[k][j]

    return graph[u][v]


# Input
graph = [
    [0, 3, INF, 7],
    [8, 0, 2, INF],
    [5, INF, 0, 1],
    [2, INF, INF, 0]
]

u = 0
v = 3

print(shortest_distance(graph, u, v))
```

```
6


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
def warshall(graph):
    n = len(graph)

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if graph[i][k] == 1 and graph[k][j] == 1:
                    graph[i][j] = 1
    return graph
# Input
graph = [
    [0, 1, 1],
    [0, 0, 1],
    [0, 0, 0]
]
# Find transitive closure
graph = warshall(graph)

# Find vertex reachable from every other vertex
n = len(graph)
answer = -1

for j in range(n):
    possible = True

    for i in range(n):
        if i != j and graph[i][j] == 0:
            possible = False
            break

    if possible:
        answer = j
        break
print(answer)
```

```
2


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
def warshall(graph):
    n = len(graph)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if graph[i][k] == 1 and graph[k][j] == 1:
                    graph[i][j] = 1
    return graph
# Input
graph = [
    [0, 1, 1, 0],
    [0, 0, 1, 0],
    [0, 0, 0, 1],
    [0, 0, 0, 0]
]
# Find transitive closure
graph = warshall(graph)

# Find vertex with maximum reachability
max_count = -1
answer = 0

for i in range(len(graph)):
    count = 0

    for j in range(len(graph)):
        if i != j and graph[i][j] == 1:
            count += 1

    if count > max_count:
        max_count = count
        answer = i

print(answer)
```

```
0


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
INF = 999999

def floyd_warshall(graph):
    n = len(graph)

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if graph[i][k] + graph[k][j] < graph[i][j]:
                    graph[i][j] = graph[i][k] + graph[k][j]

    return graph


# Input
graph = [
    [0, 2, INF],
    [2, 0, 4],
    [INF, 4, 0]
]

# Find shortest paths
graph = floyd_warshall(graph)

# Find diameter
diameter = 0

for i in range(len(graph)):
    for j in range(len(graph)):
        if graph[i][j] != INF:
            diameter = max(diameter, graph[i][j])

print(diameter)
```

```
6


...Program finished with exit code 0
Press ENTER to exit console.
```

```python
INF = 999999

def diameter(graph):
    n = len(graph)

    # Floyd-Warshall
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if graph[i][k] + graph[k][j] < graph[i][j]:
                    graph[i][j] = graph[i][k] + graph[k][j]

    # Find maximum shortest-path distance
    maximum = 0

    for i in range(n):
        for j in range(n):
            if graph[i][j] != INF:
                maximum = max(maximum, graph[i][j])

    return maximum


# Input
graph = [
    [0, 2, INF],
    [2, 0, 3],
    [INF, 3, 0]
]

print(diameter(graph))
```

```
5


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
def warshall(graph):
    n = len(graph)

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if graph[i][k] == 1 and graph[k][j] == 1:
                    graph[i][j] = 1

    return graph

# Input
graph = [
    [0, 1, 0],
    [0, 0, 1],
    [1, 0, 0]
]

# Find transitive closure
graph = warshall(graph)

# Check whether every vertex can reach every other vertex
n = len(graph)
strongly_connected = True

for i in range(n):
    for j in range(n):
        if i != j and graph[i][j] == 0:
            strongly_connected = False

if strongly_connected:
    print("Strongly Connected")
else:
    print("Not Strongly Connected")
```

```
Strongly Connected


...Program finished with exit code 0
Press ENTER to exit console.
```

```python
INF = 999999

graph = [
    [0, 5, INF, 10],
    [INF, 0, 3, INF],
    [INF, INF, 0, 1],
    [INF, INF, INF, 0]
]

n = len(graph)

# Floyd-Warshall Algorithm
for k in range(n):
    for i in range(n):
        for j in range(n):
            if graph[i][k] + graph[k][j] < graph[i][j]:
                graph[i][j] = graph[i][k] + graph[k][j]

# Print result
for row in graph:
    print(row)
```

```
[999999, 0, 3, 4]
[999999, 999999, 0, 1]
[999999, 999999, 999999, 0]


..Program finished with exit code 0
Press ENTER to exit console.
```

```python
def warshall(graph):
    n = len(graph)

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if graph[i][k] == 1 and graph[k][j] == 1:
                    graph[i][j] = 1

    for row in graph:
        print(row)


# Input
graph = [
    [0, 1, 0, 0],
    [0, 0, 1, 0],
    [0, 0, 0, 1],
    [0, 0, 0, 0]
]

# Find transitive closure
warshall(graph)
```

```
0, 0, 1, 1]
0, 0, 0, 1]
0, 0, 0, 0]
```

main.py

```python
INF = 999999
def findCity(n, edges, threshold):
    dist = [[INF] * n for _ in range(n)]
    for i in range(n):
        dist[i][i] = 0
    for u, v, w in edges:
        dist[u][v] = w
        dist[v][u] = w
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    city = -1
    min_count = INF
    for i in range(n):
        count = 0
        for j in range(n):
            if i != j and dist[i][j] <= threshold:
                count += 1
        if count <= min_count:
            min_count = count
            city = i
    return city
n = 4
edges = [
    [0, 1, 3],
    [1, 2, 1],
    [2, 3, 4],
    [0, 3, 7]
]
threshold = 4
print(findCity(n, edges, threshold))
```

```
3


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
def warshall(graph):
    n = len(graph)

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if graph[i][k] == 1 and graph[k][j] == 1:
                    graph[i][j] = 1

    return graph


# Input
graph = [
    [0, 1, 0],
    [0, 0, 1],
    [0, 0, 0]
]

u = 0
v = 2

# Find reachability
graph = warshall(graph)

if graph[u][v] == 1:
    print("Path Exists")
else:
    print("Path Does Not Exist")
```

```
Path Exists


...Program finished with exit code 0
Press ENTER to exit console.
```

```python
def warshall(graph):
    n = len(graph)

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if graph[i][k] == 1 and graph[k][j] == 1:
                    graph[i][j] = 1

    return graph


# Input
graph = [
    [0, 1, 0, 0],
    [0, 0, 1, 0],
    [0, 0, 0, 1],
    [0, 0, 0, 0]
]

source = 0

# Find transitive closure
graph = warshall(graph)

# Count reachable vertices
count = 0

for i in range(len(graph)):
    if i != source and graph[source][i] == 1:
        count += 1

print(count)
```

```
3


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
def negative_cycle(vertices, edges):
    dist = [0] * vertices

    # Bellman-Ford
    for i in range(vertices - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w

    # Check for negative weight cycle
    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            return True

    return False


# Input
vertices = 4

edges = [
    (0, 1, 1),
    (1, 2, -1),
    (2, 3, -1),
    (3, 0, -1)
]

if negative_cycle(vertices, edges):
    print("Negative Weight Cycle Exists")
else:
    print("No Negative Weight Cycle")
```

```
Negative Weight Cycle Exists


...Program finished with exit code 0
Press ENTER to exit console.
```

```python
def cheapest_flight(n, flights, src, dst, K):
    INF = 999999
    dist = [INF] * n
    dist[src] = 0

    # At most K stops means K + 1 flights
    for i in range(K + 1):
        temp = dist.copy()

        for u, v, cost in flights:
            if dist[u] != INF:
                temp[v] = min(temp[v], dist[u] + cost)

        dist = temp

    if dist[dst] == INF:
        return -1
    return dist[dst]
# Input
n = 4

flights = [
    [0, 1, 100],
    [1, 2, 100],
    [2, 3, 100],
    [0, 3, 500]
]

src = 0
dst = 3
K = 1

print(cheapest_flight(n, flights, src, dst, K))
```

```
500


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
def bellman_ford(V, edges, source):
    dist = [999999] * V
    dist[source] = 0
    for i in range(V - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            print("Negative Weight Cycle Detected")
            return
    print("Vertex Distance")
    for i in range(V):
        print(i, dist[i])
# Input
V = 5
edges = [
    (0, 1, -1),
    (0, 2, 4),
    (1, 2, 3),
    (1, 3, 2),
    (1, 4, 2),
    (3, 2, 5),
    (3, 1, 1),
    (4, 3, -3)
]
source = 0
bellman_ford(V, edges, source)
```

```
2 2
3 -2
4 1


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
INF = 999999

def network_delay(times, n, k):
    dist = [INF] * (n + 1)
    dist[k] = 0

    # Bellman-Ford
    for i in range(n - 1):
        for u, v, w in times:
            if dist[u] != INF and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w

    # Check if all nodes are reachable
    answer = 0

    for i in range(1, n + 1):
        if dist[i] == INF:
            return -1
        answer = max(answer, dist[i])

    return answer


# Input
times = [
    [2, 1, 1],
    [2, 3, 1],
    [3, 4, 1]
]

n = 4
k = 2

print(network_delay(times, n, k))
```

```
2


...Program finished with exit code 0
Press ENTER to exit console.
```